

Building a Lakehouse

Azure Databricks + Unity Catalog + ADLS Gen2

Migrating a local Excel-based analytics pipeline
to a governed, scalable, multi-tenant lakehouse — following the medallion architecture pattern.

WHO THIS IS FOR

Anyone learning modern data-platform architecture — engineers moving from Excel/Python scripts to enterprise data platforms, analysts curious about Databricks, or product folk who need to explain the lakehouse to stakeholders. No prior Databricks experience required.

Table of contents

1.	Why migrate? The problem we're solving	3
2.	What is a Lakehouse?	4
3.	Medallion Architecture (Bronze → Silver → Gold)	5
4.	Azure Databricks — the compute brain	7
5.	ADLS Gen2 — the storage backbone	8
6.	Unity Catalog — the governance layer	9
7.	Delta Lake — the table format	10
8.	The target architecture (full picture)	11
9.	Today vs Tomorrow — what changes	12
10.	Migration phases	13
11.	Cost analysis (■ per month)	14
12.	Risks, tradeoffs, when NOT to do this	15
13.	Glossary of acronyms	16
14.	Further learning	17

1. Why migrate? The problem we're solving

Today's setup works beautifully for one fitness centre with two months of data — a Python pipeline reads weekly Excel files, builds a monthly master, and a Streamlit dashboard with Claude-generated narratives runs locally. Operating cost: about ■10/month. Total people who can use it: one.

It breaks the day you want any of the following:

- **Multiple users** at the same time, from different cities, with audit trails.
- **Multiple tenants** — a SaaS offering serving many fitness centres from one platform.
- **Bigger data** — years of history × dozens of branches × millions of transactions.
- **Reliable scheduling** — automated nightly refresh with retries, alerts, restartable jobs.
- **Governance** — "who can read salary data", "prove this number's lineage to an auditor".
- **Time travel** — "what did April look like before someone overwrote the file last week?".

The shift in one sentence

We're moving from a **file-based pipeline on one laptop** to a **table-based pipeline on a governed cloud platform** — the lakehouse pattern that companies like Databricks, Microsoft, and Snowflake have converged on over the last 5 years.

2. What is a Lakehouse?

A lakehouse is a hybrid of two older patterns. To understand it, you have to know what came before.

Data Warehouse (1980s–today)

Structured tables, SQL-only, governed, fast queries — but rigid schemas, expensive storage, painful to load unstructured data. Examples: Snowflake, Azure Synapse, Amazon Redshift, traditional Oracle/SQL Server.

Data Lake (2010s)

Cheap blob storage holding anything — CSV, Parquet, JSON, images. Flexible, scales infinitely, but no transactions, no schema enforcement, hard to query without building a separate engine on top.

Lakehouse (~2020 onwards)

Pick a table format that brings warehouse features on top of cheap blob storage. You get the cost + flexibility of a lake AND the governance + speed of a warehouse. Table formats: **Delta Lake** (Databricks), **Apache Iceberg**, **Apache Hudi**.

	Data Warehouse	Data Lake	Lakehouse
Storage cost	High	Low	Low
Transactions (ACID)	Yes	No	Yes
Schema enforcement	Strict	None	Optional/Strict
File formats	Proprietary	Anything	Open (Parquet+Delta log)
Query engine	Locked-in	BYO	Many (Spark, Trino, Presto)
Handles unstructured	No	Yes	Yes
Governance	Built-in	Bolted-on	Built-in (UC)

Why this matters for our project

We have structured P&L; data (perfect for a warehouse) AND raw Excel files (messy, needs a lake). A lakehouse lets us land both, govern both, and query both — without picking one or the other.

3. Medallion Architecture

Inside the lakehouse, the medallion pattern is how you organize the data — three layers, each progressively more refined.

Bronze layer — the truth

Land everything exactly as received. No transformation. If garbage comes in, garbage lives in bronze. Why? Replayability (re-process from bronze if downstream logic has bugs), auditability (proving what the source actually said on a given date), schema-on-read.

Silver layer — the model

Take bronze and apply your business rules: dedupe, type-cast, conform to a canonical schema. Silver tables represent entities: memberships, transactions, expenses. They're query-friendly but not yet aggregated.

Gold layer — the answer

Aggregations, joins, and KPIs — the tables your dashboard and ML models actually read. One row in gold often summarizes thousands of rows in silver.

Why three layers and not one?

Each layer has a different audience and a different change frequency. Bronze is for engineers re-processing. Silver is for analysts who need clean facts. Gold is for dashboards / ML / BI. If you tried to do everything in one transform, you'd have spaghetti SQL and no clear contract between teams.

Concrete example with our data

Layer	Table	Granularity	Who reads it
Bronze	raw_weekly_files	One row per uploaded .xlsx file	Engineers debugging
Bronze	raw_ultra_rows	Source-exact membership rows	Schema explorers
Silver	memberships	One row per (member × payment date)	Analysts running SQL
Silver	expenses	One row per (month × category)	Finance team
Gold	pnl_monthly	One row per (FY × month) — wide	Dashboard, AI brief generator
Gold	member_segments	One row per (segment × month)	Marketing / sales models
AI	monthly_briefs	One row per (month × LLM run)	Dashboard reader, owner via PDF

Reading order is always **upstream** → **downstream**: silver depends on bronze, gold depends on silver. Never reverse. This dependency graph is what your scheduler (Databricks Workflows) executes.

4. Azure Databricks — the compute brain

Databricks is a managed platform that gives you a single workspace combining:

- **Compute clusters** — ephemeral or always-on Spark clusters that auto-scale
- **Notebooks** — collaborative editors mixing SQL, Python, R, Scala
- **Jobs / Workflows** — schedule + chain notebooks into pipelines with retry, alerts
- **SQL Warehouse** — serverless query endpoint that scales to zero
- **Unity Catalog** — the governance layer
- **Databricks Apps** — host a Streamlit / Dash / Gradio app inside the workspace
- **MLflow** — model registry, experiment tracking
- **Delta Live Tables** — declarative ETL framework

Workspace tiers

Tier	What you get	When to pick
Standard	Notebooks, Jobs, basic ACLs	Single-team dev environment, no governance needed
Premium	+ Unity Catalog, RBAC, SQL Warehouse, audit logs	Anything multi-tenant, regulated, or production

Our project: Premium. Unity Catalog is non-negotiable if we ever want multi-tenant SaaS.

Spark in one sentence

Spark is a distributed query engine — write a query, it gets split across many machines, results merged. For our small data Spark is overkill but Databricks bundles it for free, and the abstractions are useful even on small data.

5. ADLS Gen2 — the storage backbone

Azure Data Lake Storage Gen2 is Azure's purpose-built storage for analytics. It's Azure Blob Storage with two extras turned on:

- **Hierarchical Namespace (HNS)** — real directories instead of flat key prefixes.
- **POSIX-style ACLs** — per-folder, per-user permissions instead of container-level SAS tokens.

Structure for our project

abclakehouse (storage account)

- landing/ ← weekly .xlsx files dropped here
- bronze/ ← Auto Loader writes Delta tables here
- silver/ ← cleaned Delta tables
- gold/ ← business-ready Delta tables
- ai/ ← LLM-generated markdown artifacts
- checkpoints/ ← streaming job state

Why not just a Windows file share?

ADLS gives us: **99.9% durability**, **geo-redundancy** if you want it, **petabyte scale**, **integrated identity** (Azure AD), **encryption at rest**, and **~\$2 per GB per month**. A file share gives us none of those.

Access from Databricks

Databricks accesses ADLS Gen2 via **OAuth + Azure Service Principal**, configured through Unity Catalog's **External Locations**. No keys stored in notebooks. The service principal is granted Storage Blob Data Contributor on the container, Databricks does the rest.

6. Unity Catalog — the governance layer

Unity Catalog (UC) turns Databricks into a proper data platform. Three things to know:

1. The three-level namespace

Every object in UC has a three-part name: `catalog.schema.object`. Think of it as: company . department . file.

catalog	Top-level container — usually per environment or business unit. We'd have <code>abc_lakehouse</code> (prod), <code>abc_lakehouse_dev</code> (dev), maybe <code>tenant_b_lakehouse</code> (a different tenant).
schema	What used to be a 'database' in older systems. In our project: <code>bronze</code> , <code>silver</code> , <code>gold</code> , <code>ai</code> .
object	Table, view, function, volume. For example: <code>abc_lakehouse.gold.pnl_monthly</code> .

2. Centralized access control

Permissions are granted at any level — catalog, schema, or object — using standard SQL. Grants are inherited downward.

- `GRANT SELECT ON SCHEMA abc_lakehouse.gold TO `analysts`` — analysts can read every gold table
- `GRANT USAGE ON CATALOG abc_lakehouse TO `dashboard-svc-principal`` — service principal can browse the catalog
- `REVOKE SELECT ON TABLE abc_lakehouse.silver.salaries FROM `interns`` — protect sensitive columns

3. Lineage and audit, for free

Every read and write through UC is logged. Click on a gold table and you get a visual graph showing which silver tables feed it, which bronze tables feed those, and which dashboards/notebooks consume it.

The 'before UC' world

Pre-Unity Catalog, permissions were per-workspace, lineage was manual, and cross-workspace sharing required exporting + re-importing data. UC fixed all of this. If anyone proposes a Databricks design without Unity Catalog in 2025, push back hard.

7. Delta Lake — the table format

Delta Lake is the file format that makes the lakehouse work. Under the hood, a Delta table is just a folder containing two things:

- **Parquet files** with the actual row data (compressed, columnar)
- A **_delta_log/** sub-folder with JSON transaction logs describing every commit.

What you get because of the transaction log

ACID transactions	Two writers can't corrupt each other. A reader sees a consistent snapshot. Same guarantees as Postgres.
Time travel	Query the table as of yesterday: <code>SELECT * FROM pnl_monthly VERSION AS OF 47</code> or <code>TIMESTAMP AS OF '2026-04-30'</code> . Free audit + free undo.
Schema enforcement	Writes that don't match the table's declared schema are rejected.
Schema evolution	Add a column without rewriting history. Old rows get NULL in the new column.
MERGE / UPSERT	Single SQL statement to insert-or-update — what data lakes famously couldn't do.
Streaming-friendly	A Delta table can be both a sink and a source for streaming jobs.

Why this is a big deal

Before Delta, querying a data lake meant either: (a) writing complex Spark code to handle eventual consistency, or (b) loading into a warehouse first. Delta closed that gap — now SQL on raw cloud storage behaves like SQL on a real database, at 10-100x lower cost.

8. The target architecture

Component summary

ADLS Gen2	Storage for raw files + Delta tables. Single source of truth.
Azure Databricks (Premium)	Compute, Unity Catalog metastore, Jobs/Workflows, SQL Warehouse, hosting for the Streamlit dashboard via Databricks Apps.
Unity Catalog	Governs every catalog, schema, table, and grant.
Auto Loader	Built-in utility — incrementally ingests new files as they arrive.
Databricks Workflows	Scheduler — visual DAG, retries, alerts.
SQL Warehouse	Serverless SQL endpoint. Auto-scales to zero when idle.
Databricks App	Streamlit dashboard hosted inside Databricks.
Anthropic API	Same Claude Haiku 4.5, called from a Databricks notebook job.

9. Today vs Tomorrow — what changes

Today (local Python)	Tomorrow (lakehouse)
Weekly .xlsx in 1.Ingestion/	Weekly .xlsx in ADLS landing/ container
pipeline.py uses Excel COM	Auto Loader → notebook reads with PySpark or pandas-on-Spark
ABC Master.xlsx (Gold)	abc_lakehouse.gold.pnl_monthly (Delta table)
pnl_reader.py with openpyxl	databricks.sql.connect() → SELECT * FROM gold.pnl_monthly
narrate_month.py CLI	Databricks notebook scheduled monthly
sentinel.py --all	Databricks notebook scheduled daily
AI- Output/*.md files	abc_lakehouse.ai.monthly_briefs (markdown stored as STRING)
refresh_all.py orchestrator	Databricks Workflow (visual DAG)
Streamlit on localhost:8501	Streamlit hosted as Databricks App, public URL via Azure AD
.env on disk	Databricks Secret Scope
data_sources.py (Prod/Dev switch)	Two catalogs: abc_lakehouse vs abc_lakehouse_dev
Single user (you)	Multi-user with per-row ACLs via UC
No history	Time travel built-in (every Delta commit is a recoverable version)
~\$10/month	~\$1,500–3,000/month at idle, scales with use

10. Migration phases

Never migrate everything at once. Break into phases, each independently valuable, each independently rollback-able.

Phase	What gets built	Est.
P1 — Foundation	Azure subscription resources: ADLS Gen2 account, Databricks workspace (Premium), Unity Catalog metastore, service principals, secret scopes. Create the catalog + empty schemas.	1 day
P2 — Bronze + Silver	Auto Loader notebook to ingest landing/*.xlsx into bronze tables. PySpark cleaner turning bronze into silver. Schedule via Workflow.	1–2 days
P3 — Gold tables	SQL views/materialized tables that produce dashboard inputs: pnl_monthly, member_segments, etc. Mirrors what your current Python code computes.	1 day
P4 — AI Jobs	Convert narrate_month, sentinel, narrate_year into Databricks notebooks. Each writes output to ai.* tables. Schedule via Workflow.	2 days
P5 — Dashboard on Databricks Apps	Modify pnl_reader to use Databricks SQL Connector against gold.*. Deploy the Streamlit app. Bind to SQL Warehouse.	1 day
P6 — Governance + monitoring	Grant per-role access. Enable Lakehouse Monitoring on critical gold tables. Wire alerts to email/Slack.	1 day
P7 — Production cutover	Switch the dashboard URL from localhost to the Databricks Apps URL. Decommission the local refresh job. Document the runbook.	0.5 day

Total: 7–9 working days for a single developer who knows Python/SQL but is new to Databricks. Allow 2x if you're learning from scratch.

11. Cost analysis

Databricks pricing is two-axis: **compute** (DBU per hour) + **storage** (Azure ADLS Gen2 rates). DBU is roughly normalized work per hour.

Component	Pricing model	Estimated £/mo (our workload)
ADLS Gen2 storage	~£2 per GB-month	£2 (data is small)
Databricks Premium workspace	No fixed fee	£0
Jobs cluster (Standard, single-node)	~£12/hour	£100–200 (10–15 hr/mo)
SQL Warehouse (Serverless 2X-Small)	~£35/hour, scales to zero	£200–500
Databricks Apps	~£400/mo for Small always-on	£400
Anthropic API (Claude Haiku 4.5)	Pay per token	£20
TOTAL (low usage, single fitness centre)		~£700–1,200
TOTAL (moderate, 10 tenants)		~£2,500–4,500
TOTAL (heavy, 100 tenants)		~£15,000–30,000

Cost reality for one fitness centre

£700–1,200/month is ~100× more expensive than today's £10. This is the price of governance, multi-user access, and the ability to add tenants. **Only worth it if you actually plan to grow beyond one user.**

12. Risks, tradeoffs, when NOT to do this

Where this design wins

- Multi-tenant SaaS for many fitness centres — one platform, isolated data per tenant
- Compliance / audit needs — UC lineage is a one-click audit answer
- Teams of analysts/engineers — proper schema, governed access
- Growing data — works on 1 GB or 1 TB with the same code
- BI tool integration — Power BI, Tableau, Looker all plug into SQL Warehouse

Where this design loses

- **One user, one machine** — paying █1,000+ to replicate █10 of local Streamlit.
- **Tiny data forever** — if your data will never exceed 100 MB, even a Postgres container on a VPS is overkill.
- **Strict data-residency** — on-prem solutions might fit better.
- **Team not ready for SQL + Python** — train first or simplify the stack.

Risks to plan for

Runaway costs	Set up Azure Cost alerts on day 1 with a hard limit. An always-on classic SQL warehouse can burn █50K/month overnight.
Vendor lock-in	Delta Lake itself is open source — readable with engines other than Databricks. But Workflows, Apps, and UC metastore are Databricks-specific.
Schema-drift in bronze	Set up Lakehouse Monitoring on critical tables.
PII in bronze	Bronze keeps the raw source. UC permissions on bronze must be tighter than gold, not looser.

13. Glossary of acronyms

ACID	Atomicity, Consistency, Isolation, Durability — properties of a real transaction.
ADLS Gen2	Azure Data Lake Storage Generation 2.
Auto Loader	Databricks utility for incremental file ingestion.
Azure AD	Azure Active Directory — Microsoft's identity service (now Entra ID).
Bronze / Silver / Gold	The three layers of the medallion architecture.
DBU	Databricks Unit — the unit of work for compute pricing.
DAG	Directed Acyclic Graph — the shape of a pipeline.
Delta Lake	Open-source table format with ACID, time travel, schema enforcement.
Delta Live Tables	Databricks' declarative ETL framework.
ETL / ELT	Extract-Transform-Load vs Extract-Load-Transform.
HNS	Hierarchical Namespace — the feature in ADLS Gen2.
Lakehouse	Architecture combining lake storage with warehouse features.
Lineage	The chain of tables/transformations producing an output.
Managed table	A table whose storage location is managed by UC.
Medallion architecture	The bronze/silver/gold pattern.
MERGE	SQL operation that combines INSERT and UPDATE.
Parquet	Open columnar file format. Delta stores its data as Parquet.
PII	Personally Identifiable Information.
Service Principal	An Azure AD identity for a service or application.
Spark	Apache Spark — distributed compute engine.
SQL Warehouse	Databricks' serverless SQL endpoint.
Time travel	Querying a Delta table at a past version or timestamp.
Unity Catalog (UC)	Databricks' governance layer.
Volume	A UC object representing a folder of unstructured files.

Workflow / Job

Databricks' scheduler.

14. Further learning

Free official resources

- **Databricks Academy** — docs.databricks.com/learn. Free courses + certifications.
- **Databricks free trial** — 14 days of full Premium workspace.
- **Delta Lake documentation** — docs.delta.io.
- **Microsoft Learn for Azure Databricks** — learn.microsoft.com.
- **Unity Catalog concepts** — docs.databricks.com/data-governance/unity-catalog.

Books worth your time

- **The Data Engineer's Handbook** — Joe Reis & Matt Housley.
- **Delta Lake: The Definitive Guide** — O'Reilly (free from Databricks).
- **Fundamentals of Data Engineering** — same authors. The lakehouse chapter is gold.

If you only do one thing

Sign up for the Databricks free trial, follow the 30-minute medallion-architecture tutorial. You'll have all the concepts from this guide running on real infrastructure before lunch.

End of guide. Generated 26 May 2026 · Self-study & teaching reference · MIT licence.